

Project Proposal - OpenPark

Team01

- 라이선스: MIT
- 기간: 2026. 05. 01. - 2026. 06. 09.

1. 개요

추진 배경

최근 교통 연구 및 관련 보도에 따르면, 도심 교통체증의 약 20~30%는 주차 공간을 찾기 위해 배회하는 차량에서 발생하며 이는 막대한 사회적 비용을 초래한다. 이를 해결하고자 지자체별로 통합 주차 정보 플랫폼을 운영하고 있으나, 대부분 공영 주차장 정보에만 국한되어 있어 실효성 측면에서 한계가 있다는 지적을 꾸준히 받고 있다.

이에 본 프로젝트 'OpenPark'는 고가의 전용 장비 없이 소규모 민간 주차장에서도 쉽게 도입할 수 있는 오픈소스 형태의 스마트 주차 관리 서비스를 제안한다. 본 시스템은 번호판 인식 클라이언트, 주차장 관리 서버, 통합 정보 허브 서버, 프론트엔드로 구성되며, 각 컴포넌트는 독립적으로 교체 및 확장이 가능하도록 유연하게 설계된다.

특히, 초기 단계에서는 접근성이 높은 일반 노트북 웹캠으로 클라이언트를 구현한다. 클라이언트 단에서 영상을 자체 처리한 후 서버에는 파싱된 번호판 정보만 전송하는 구조를 채택함으로써 네트워크 부하를 최소화했고, 이러한 설계는 라즈베리파이 등 향후 클라이언트의 규격 및 구조 변경을 용이하게 한다.

목표

- 소규모 주차장을 운영하는 관리자들이 쉽게 사용할 수 있는 오픈소스 기반의 주차장 관리 시스템 제공 및 사설 서버 구축이 어려운 관리자를 위한 Public 서버 운영
- 고가의 전용 장비 없이 일반 웹캠과 오픈소스를 활용하여 소규모 민간 주차장의 시스템 도입 장벽 완화
- 정보의 사각지대에 있던 소규모 민간 주차장의 입출차 기록 및 실시간 잔여 공간 데이터를 전산화
- 민영 및 공영 주차장 현황 제공 플랫폼 운영

내용

- **OpenCV 및 YOLO 기반의 번호판 인식 시스템 구현**
AI 모델 기반 번호판 인식 시스템을 구현하고 로컬 혹은 공용 서버 백엔드로 번호 정보를 전송하는 시스템 구축. 더불어, PC 환경에서의 초기 검증 후 라즈베리파이(Edge Device) 환경에 맞춰 AI 모델을 경량화하고 이식하여 하드웨어 확장성 검증
- **Private/Public Backend Server 구축 및 Parking Lot Information Hub 운영**
자체 서버 구축 역량이 있는 관리자에게는 보안이 강화된 독립적인 Private 서버 환경을, 인프라 관리가 어려운 관리자에게는 즉시 도입 가능한 Public 서버를 제공하여 시스템 접근성을 극대화. 또한, 수집된 민간 주차장 데이터를 공용 주차장 OpenAPI 데이터와 결합하여, 지역 내 전체 주차 현황을 통합하고 라우팅하는 Parking Lot Information Hub 운영
- **사용자 친화적 통합 주차 공간 확인 웹 서비스 / API 제공**
주차장 정보가 필요한 사용자들을 위해 웹 서비스 및 API를 구축하여, 편리하게 주차 공간을 확인할 수 있도록 지원

2. 요구사항

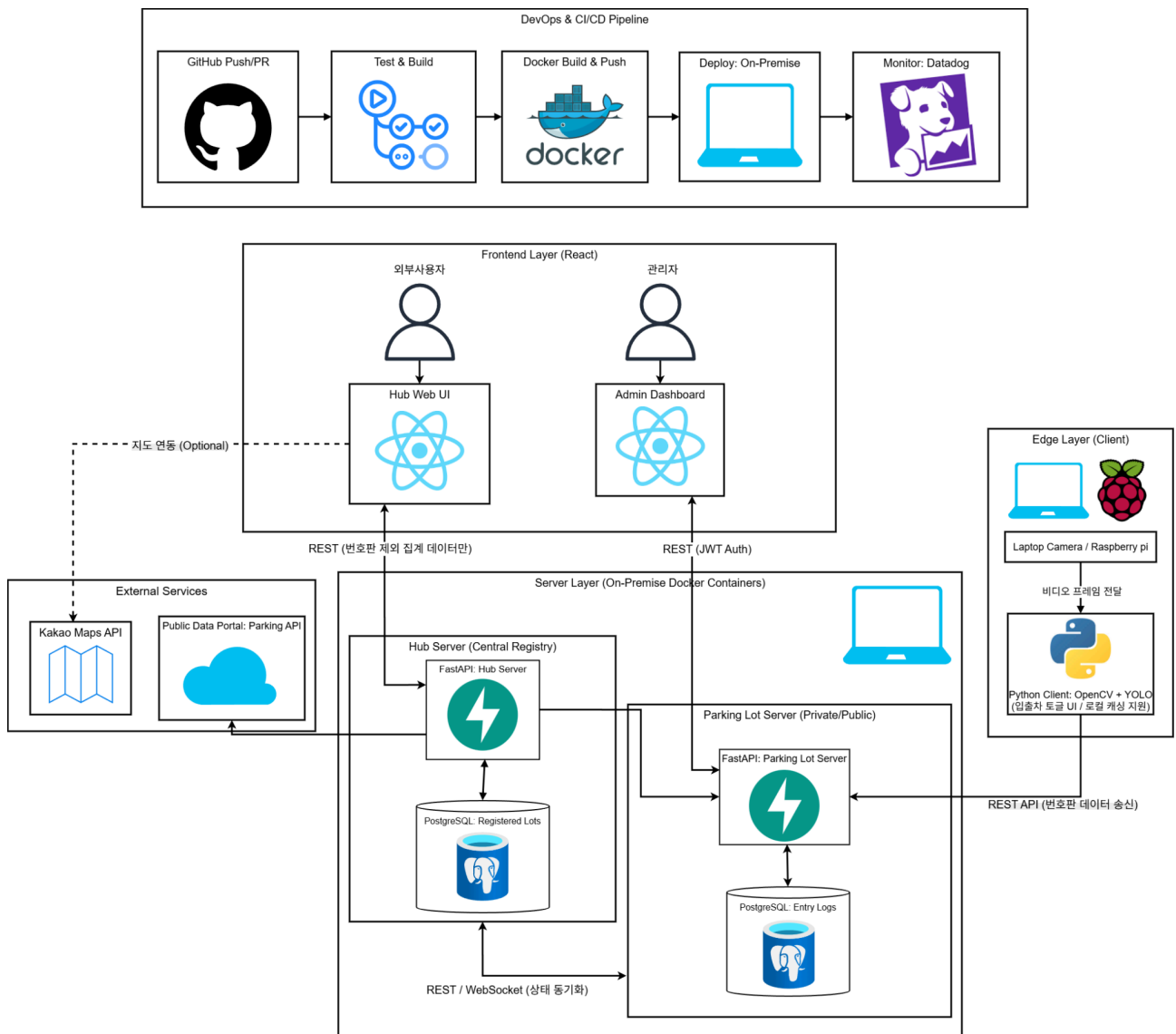
Functional Requirements

- 번호판 자동 인식
- 입출차 기록 DB 저장
- 관리자 로그인 및 인증
- 주차장 및 사용자 등록 기능
- 공용 주차장 OpenAPI 연동 및 포맷 통일
- 민간/공용 주차장 통합 현황 목록 UI
- 지도 API 연동

Non-Functional Requirements

- Vision AI 번호판 인식률 90% 이상 달성
- 개인정보 관리를 위해 차량 번호판은 외부에 절대 노출하지 않음
- 온프레미스 환경에서 직접 운영 가능한 Private API Server 제공
- Client를 제외한 각 컴포넌트는 Docker로 독립 실행
- PC 환경 검증 이후, 라즈베리파이 환경으로도 클라이언트 이식 가능하도록 설계

3. 시스템 구조



- **Deploy:** Frontend, Public Parking Lot Server, Hub Server는 **온프레미스** 환경에 배포 및 운영
- **CI/CD:** GitHub main 브랜치 PR 발생 시 자동으로 테스트, 빌드, Docker 이미지 생성, 배포 진행
- **Frontend:** 사용자용 Web UI 및 관리자 Dashboard 제공
- **Client:** 차량 번호판 수집 및 추출 후 Parking Lot Server에 전송
- **Parking Lot Server:** 입출차 데이터 처리 및 실시간 상태 제공
- **Hub Server:** 민영, 공영 주차장 정보 중앙 관리

4. 구성요소

4-1. Client

번호판 인식 및 서버 전송을 담당한다. 단일 클라이언트 프로그램 내에 '입출차 모드 토글 UI'를 구현하여, 한 대의 카메라에서 입차 또는 출차를 선택적으로 병렬 처리한다.

- OpenCV + YOLO (한국 번호판 fine-tuned 모델) 로 번호판 파싱
- 파싱된 번호판, 입출차 타입, 주차장 ID, 타임스탬프를 Parking Lot Server로 송신
- Python GUI로 현재 인식 상태 실시간 확인
- API 서버 주소 설정으로 Private/Public 모드 전환
- Docker 미사용 (카메라 디바이스 접근 문제)
- PC 웹캠을 통한 초기 기능 검증 후, 라즈베리파이 환경에 맞춰 AI 모델을 경량화하여 구축

기술스택: Python, OpenCV, YOLO

4-2. Parking Lot Server

클라이언트로부터 번호판 데이터를 수신하고 DB에 기록한다. 관리자 UI에 실시간 데이터를 제공한다.

- 입출차 기록 (번호판, 타입, 시각) DB 저장
- 잔여 공간 및 차량 목록 제공
- WebSocket으로 잔여 공간 실시간 업데이트
- 주차장 등록 및 사용자 등록 기능

기술스택: FastAPI, PostgreSQL, Docker

4-3. Hub Server

외부 사용자에게 민간/공용 주차장 통합 정보를 제공하는 프록시 서버다.

- 민간 주차장 요청 → Parking Lot Server Public API로 라우팅
- 공용 주차장 요청 → OpenAPI(대구광역시 통합주차정보시스템)로 라우팅

- 두 소스의 응답 포맷을 통일하여 단일 API로 제공 (Adapter 레이어)
- 차량 번호는 절대 외부 노출 없이 총 공간/잔여 공간만 제공
- 등록된 주차장 ID 및 메타정보 관리

기술스택: FastAPI, PostgreSQL, Docker

4-4. Frontend

Parking Lot Server 관리자 UI

- 입출차 현황 리스트
- 잔여 공간 실시간 표시
- 주차장 관리 Dashboard

Hub Server UI

- 민간/공용 주차장 통합 목록 및 잔여 공간 현황
- 지도 API 연동 시각화

기술스택: React, Docker

5. Frameworks & Technologies

컴포넌트	기술스택
Client	Python, OpenCV, YOLO, (Raspberry Pi)
Parking Lot Server	FastAPI, PostgreSQL, Docker
Hub Server	FastAPI, PostgreSQL, Docker
Frontend	React, Docker
Deploy	사설 서버 예정
Map	카카오맵 API (검토 중)
Parking Data	대구광역시 통합정보시스템

6. MVP 목표

1차 목표

- 번호판 인식 및 입출차 자동 기록 (인식률 90% 이상)
- Parking Lot Server 입출차 기록 및 잔여 공간 관리
- 관리자 확인 UI (단순 리스트)

2차 목표

- Hub Server 구축 및 공용 주차장 OpenAPI 연동
- 민간/공용 주차장 통합 현황 목록 UI
- 라즈베리파이 환경으로 클라이언트 이식 및 AI 모델 경량화

3차 목표

- 지도 API 연동으로 실시간 주차장 현황 시각화

7. 프로젝트 관리 및 협업

- **애자일 기반 협업**

2주 단위 스프린트로 프로젝트 진행, 각 스프린트마다 MVP 단위 결과물 도출
총 3회의 스프린트를 통해 점진적 기능 완성 및 개선

- **GitHub Projects 활용**

칸반 보드(Todo / In Progress / In Review / Done) 기반으로 작업 흐름 시각화
각 태스크에 담당자, 진행도, 서브태스크를 명확히 기록하여 진행 상황 관리

- **GitHub Issues 기반 작업 관리**

Issue 생성 후 개발을 시작하는 Issue-Driven 방식 적용
Github Projects에 있는 태스크를 기반으로 Issue를 자동으로 생성
기능, 버그, 문서 작업 등을 라벨링하여 작업 유형 구분

- **브랜치 전략**

GitHub Flow 기반 운영
main 브랜치는 항상 배포 가능한 상태 유지, 직접 푸시 금지
feature/{기능명} 또는 fix/{버그명} 브랜치에서 독립적으로 개발 진행

- **Pull Request 및 코드 리뷰**

모든 변경 사항은 PR을 통해 main 브랜치에 병합
자동화된 Test와 Build를 통해 코드가 정상적으로 동작하는지 확인
최소 1인 이상의 코드 리뷰 승인 후 Merge 진행
PR 템플릿을 활용하여 작업 내용, 테스트 결과, 리뷰 포인트 명시

8. 주차별 계획

주차	기간	목표
1주	5/1 - 5/7	Proposal 제출, 시스템 구조도 확정, DB 스키마 합의, GitHub 프로젝트 세팅
2주	5/8 - 5/14	Client 번호판 인식 구현, Parking Lot Server API 기초 구현
3주	5/15 - 5/21	1차 목표 통합 테스트, 관리자 UI 기초 완성, 라즈베리파이 환경 세팅 및 Client 코드 1차 이식
4주	5/22 - 5/28	라즈베리파이 엣지 환경에 맞춘 AI 모델 경량화 및 최적화, Hub Server 구축, 공용 OpenAPI 연동, 2차 목표 착수
5주	5/29 - 6/4	2차/3차 목표 완성, 버그 수정
6주	6/5 - 6/9	하드웨어(라즈베리파이) 최종 현장 테스트 및 세팅 가이드 작성, 최종 발표 준비, 데모 정리

9. 구성원 및 역할분담

성명	학번	역할
김준석	2021111675	Client CV/AI, Backend
김태이	2024017774	Client CV/AI, Frontend
박수겸	2020112387	System Architecture Design, Backend
심준성	2021111589	Frontend, CI/CD, GitHub Project Management